

MIGRATION GUIDES

MIGRATING FROM GO TO RUST



by Matthias Endler  Last updated: 2026-07-20

Out of all the migrations I help teams with, Go to Rust is a bit of an outlier. It's not a question of "is Rust faster?" or "does Rust have types?", Go already gets you most of the way there. The discussion is mostly about **correctness guarantees**, **runtime tradeoffs**, and **developer ergonomics**.

A quick disclaimer before we start: this guide is **heavily backend-focused**. Backend services are where Go is strongest, small static binaries, a standard library focused on networking, and an ecosystem of libraries for HTTP servers, gRPC, databases, etc.

That's also where most teams considering Rust are coming from (at least the ones who reach out to me), so I think that's the comparison that's actually useful in practice. If you're writing CLI tools, embedded firmware, or game engines, some of this still applies, but to be honest, I'm afraid this is not the best resource for you.

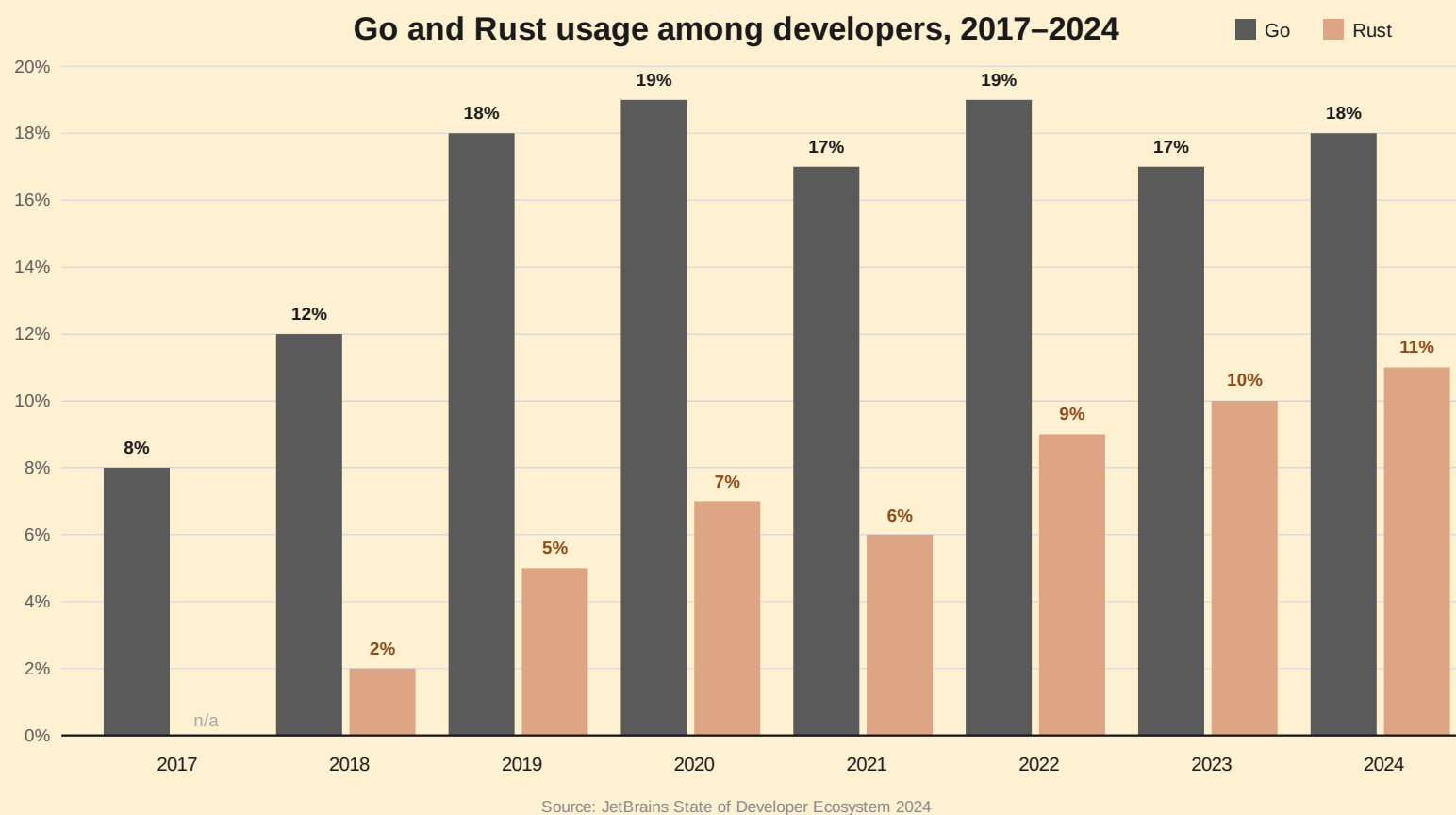
For context, I've written about Go and Rust before: ["Go vs Rust? Choose Go."](#)  back in 2017, and later the ["Rust vs Go: A Hands-On Comparison"](#)  with the Shuttle team, which walks through a small backend service in both languages.

What you will learn in this article

- Where Go and Rust overlap, and where they diverge.
- How Go patterns map to Rust.
- What you gain from the borrow checker.
- Where I tell people to keep Go and where Rust is worth the migration cost.
- How to migrate Go services incrementally.

WHERE I'M COMING FROM

I'll be upfront: I'm not a fan of Go. I think it's a *badly designed* language, even if a very successful one. It confuses *easiness* with *simplicity*[↗], and several of its core design tradeoffs (`nil` everywhere, error handling as a discipline rule rather than a type, the long absence of generics) point in a direction I disagree with. That said, success matters! Go has captured a real and persistent share of working developers, hovering around 17–19% in the JetBrains Developer Ecosystem Survey. Rust is growing steadily but is still a smaller slice:



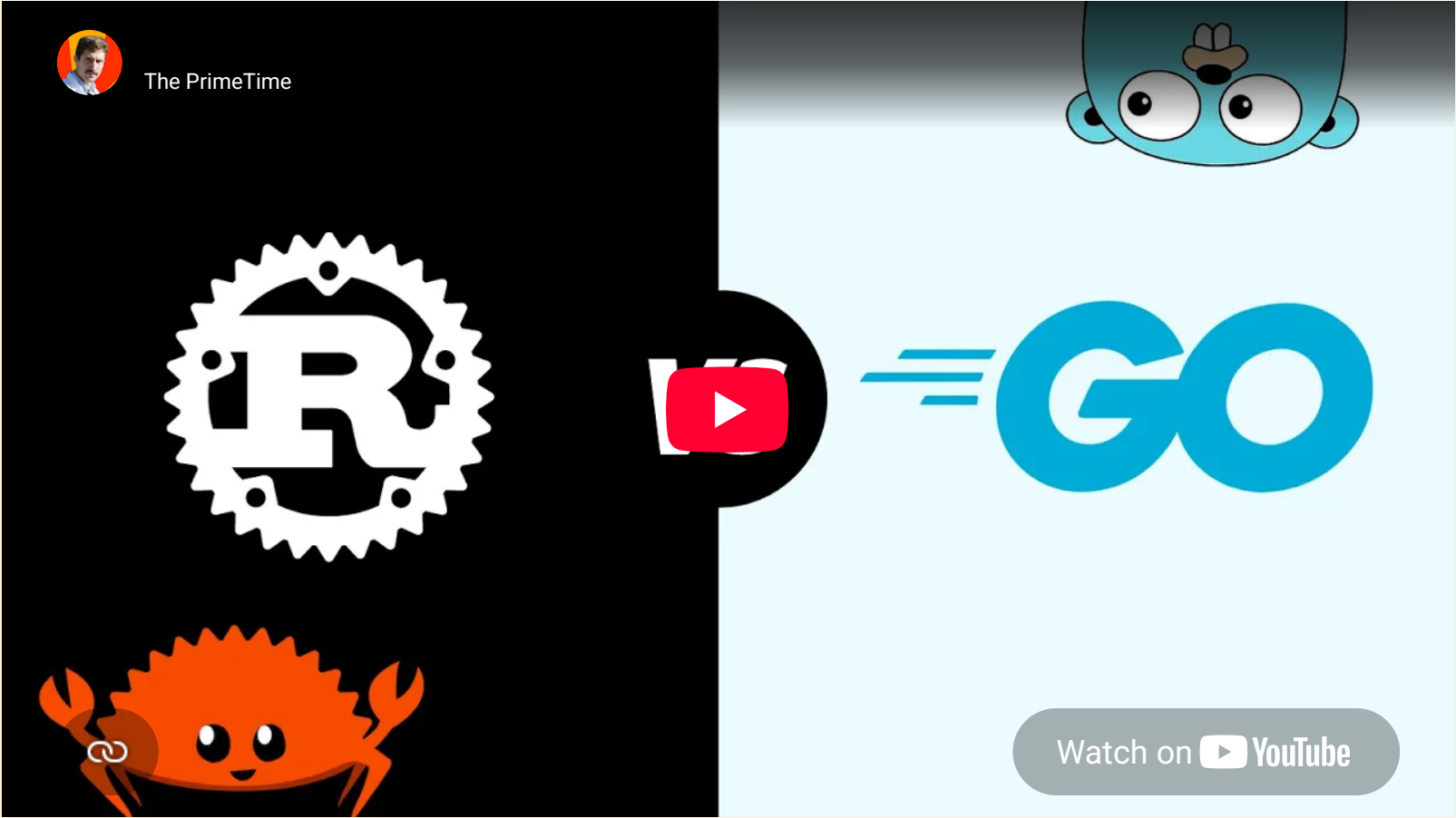
Go is clearly working for a lot of people, and a guide that pretends otherwise isn't helpful. So I'll do my very best to be objective in this guide rather than relitigate old arguments. But you should know my priors so you can calibrate.

The other prior worth disclosing: I run a Rust consultancy; of course I'm biased! More people using Rust is good for my business. But I've also worked in both languages professionally and shipped Go services to production.

This guide is for Go developers who want an honest, side-by-side look at what changes when you move to Rust.

For a deliberately opposite take, I recommend reading [“Just Fucking Use Go”](#)[↗] by Blain Smith. Holding both views in your head at once is more useful than either one alone.

If you prefer to watch rather than read, here's a video from the Shuttle article above, read and commented by the Primeagen:



A FIRST LOOK AT THE MOST IMPORTANT COMMANDS

Go developers already have one of the cleanest toolchains in the industry. Back in the day, it helped start the trend of “batteries included” toolchains: one consistent interface for building, testing, formatting, and dependency management. I’m glad Rust followed suit. It’s one of my favorite parts about both ecosystems.

`cargo` has even more built-in:

Go tool	Rust equivalent	Notes
<code>go build</code>	<code>cargo build</code>	Compile the project
<code>go run .</code>	<code>cargo run</code>	Build and run
<code>gofmt</code> / <code>goimports</code>	<code>cargo fmt</code>	Auto-formatter, zero config
<code>go test ./...</code>	<code>cargo test</code>	Testing built into the toolchain
<code>go vet ./...</code>	<code>cargo clippy</code>	Linters; Clippy catches more style and correctness issues than <code>vet</code>
<code>go install ./cmd/foo</code>	<code>cargo install --path .</code>	Install a binary
<code>golangci-lint run</code>	<code>cargo clippy -- -D warnings</code>	Strict lint mode
<code>go doc</code>	<code>cargo doc</code>	Generate and view API docs
<code>pprof</code>	<code>cargo flamegraph</code> / <code>sample</code>	CPU profiling
<code>govulncheck</code>	<code>cargo audit</code>	Vulnerability scanning against an advisory database

The big difference is that in Go you typically reach for third-party tools (`golangci-lint`, `mockgen`, `air`, `goreleaser`) to fill gaps. In Rust, the first-party ecosystem covers more out of

the box. To be precise, `cargo audit` and the profilers (`cargo flamegraph`/`samply`) are community crates, not part of cargo itself. But they, along with the other external tools (`cargo watch`/`bacon` [↗](#), `cargo nextest` [↗](#)), install with one command and feel native, e.g. `cargo install cargo-nextest` gives you `cargo nextest` right away.

Both communities have converged on the same insight: a single canonical formatting style (even if imperfect!) is worth more than the bikeshedding it eliminates.

Gofmt's style is no one's favorite, yet gofmt is everyone's favorite.

— Rob Pike, [Go Proverbs](#) [↗](#)

The same is true of `rustfmt`. Not everyone likes every detail, but removing style debates from code review is worth more than the occasional formatting choice you'd have made differently.

KEY DIFFERENCES BETWEEN GO AND RUST

The main differences between Go and Rust are about **what guarantees you get from the compiler** and **how much control you have over runtime behaviour**.

Go and Rust are both statically typed, compiled languages with strong concurrency stories, but they diverge on what the compiler guarantees. Go leans on a garbage collector, runtime race detection, and `if err != nil` conventions. Rust pushes memory management, data-race prevention, and error handling into the type system via ownership, `Send`/`Sync`, and `Result<T, E>`.

The practical tradeoff is that Go gives you a gentle learning curve, very fast compile times, and a larger ecosystem, whereas Rust gives you no GC, stricter compile-time checks, and zero-cost abstractions at the cost of a steeper learning curve and slower builds.

Most of what changes when you move from Go to Rust is that checks get pulled into the type system. Nil-handling, error propagation, data races, resource lifetimes, cancellation, and generics are things Go relies on convention, tooling (`go vet`, `errcheck`, `golanci-lint`, `-race`), or runtime detection. Rust encodes them as types the compiler enforces directly.

Does that mean “more cognitive overhead”? I'd challenge that. It's *more* upfront, yes, but it's also *harder to hold wrong*. A `Mutex<T>` in Rust doesn't just document that the data needs a lock, it makes the lock the *only* way to reach the data: you call `.lock()`, you get a guard, and the guard is what gives you access to the inner value. Drop the guard and the lock releases automatically. There is no “I forgot to lock” path because the unlocked path doesn't exist in the type. Once you internalize that pattern, and you find it repeated everywhere (`Option`, `Result`, `&mut T`, `Send`/`Sync`, RAIL guards), Rust stops feeling heavy and starts feeling like the compiler is doing work you used to do in your head.

❗ It's Not About The Runtime!

People often claim that a managed runtime is “good enough for most backends”, but I think they are missing the point. In my opinion, Go optimizes for quick iteration speed while Rust optimizes for correctness. More control over memory is a nice side effect for most production workloads. It can mean fewer machines for the same work, but the main reason to choose Rust is still correctness.

REASONS WHY TEAMS CONSIDER MOVING FROM GO TO RUST

Go developers don't usually come to Rust because Go is “too slow.” For most backend workloads, Go is plenty fast!

What people tell me when I ask them why they're looking at Rust is that they got frustrated by the many smaller pain points that add up: Go's verbose error handling, the danger of segmentation faults from `nil` pointers, and the lack of generics (for a long time) or any sophisticated type system features, such as enums or traits. The Go standard library has some weird gaps, such as the lack of a `Set` type. (The idiomatic workaround is `map[T]struct{}`, which works fine in practice but isn't exactly equivalent to a first-class set type.)

`nil` PANICS IN PRODUCTION

You ship a Go service, which runs fine for months but then a code path runs where someone forgot to check whether a pointer was `nil`, and the goroutine panics. A common case is a lookup that returns the zero value, or a struct whose pointer fields survived deserialization without being populated:

```
func (s *Service) Handle(req *Request) error {
    // Find returns (*User, error). The error is nil for "not found";
    // the caller is expected to check user != nil, but this is very easy to forget.
    user, err := s.repo.Find(req.UserID)
    if err != nil {
        return err
    }
    return user.Account.Notify() // crashes if user is nil, or if Account is nil
}
```

Linters and IDE checks catch *some* of these (`nilaway`, `staticcheck`), but they're opt-in, best-effort (they miss cases rather than proving absence), and don't cross package boundaries reliably. Go's compiler itself does not force you to consider the absence case, but Rust's `Option<T>` does:

```
fn handle(&self, req: &Request) -> Result<(), ServiceError> {
    // find returns Option<User>; ok_or turns None into a typed error, then ? propagates
    let user = self.repo.find(req.user_id).ok_or(ServiceError::NotFound)?;
```

```
user.notify()
}
```

You literally **cannot** dereference an `Option` without acknowledging the `None` case. Whole categories of pager-duty incidents disappear. 😏¹

-race **WON'T CATCH ALL DATA RACES**

`go test -race` is a great tool, but it's a runtime detector, it only finds races that *actually execute* during testing. Mutating a map from two goroutines without a lock compiles fine in Go and only blows up in production under load.

In Rust, sharing mutable state across threads requires types that implement `Send` and `Sync`. Try to share a plain `HashMap` between threads and **the program does not compile**. You're forced to wrap it in an `Arc<Mutex<...>>`, an `Arc<RwLock<...>>`, or use a channel. That means data races become a compile-time error. Rust won't save you from every concurrency bug, though.²

In our interview, Paul Dix has been very candid about what motivated the InfluxDB Go to Rust rewrite:

[The main benefit is] fearless concurrency — eliminating data races essentially, which we had before. Really gnarly bugs in version 1 of Influx due to that.

— Paul Dix, Founder & CTO, InfluxData, on Rust in Production

COMPOSABLE ERROR HANDLING

`if err != nil { return err }` is fine... for a while. After a few years, you notice three things:

1. The boilerplate dilutes the actual logic of your function.
2. Wrapping with `fmt.Errorf("doing X: %w", err)` is an exercise in discipline. If you forget, you leave valuable context on the floor.
3. Sentinel errors via `errors.Is`/`errors.As` work, but the compiler doesn't tell you when you forgot to handle a new variant.

It's worth being honest about the counter-argument here, since it came up in the Lobste.rs thread [↗](#) on my Shuttle article: experienced Go developers point out that `errcheck` and `golanci-lint` catch most of the "forgot to handle the error" cases in practice, and that explicit `if err != nil` is *easier to read* than dense `?` chains. Both points are fair, and the explicit style is a deliberate cultural value:

I think that error handling should be explicit, this should be a core value of the language.

— Peter Bourgon, GoTime #91 [↗](#), quoted in Dave Cheney's Zen of Go [↗](#)

I would agree with that, even in Rust. In my mind, `?` is also explicit. At least I don't know anyone who's worked with Rust for any length of time and wouldn't consider `?` to be a clear signal that a function can fail. The "boilerplate-vs-readability" tradeoff is quite subjective.

In Rust, you'd put all the error variants into one place and let the type system handle the conversion:

```
#[derive(Debug, thiserror::Error)]
pub enum UserError {
    #[error("user {0} not found")]
    NotFound(UserId),
    #[error("user already exists")]
    AlreadyExists,
    #[error(transparent)]
    Repo(#[from] RepoError),
}

pub fn rename(id: UserId, name: &str) → Result<User, UserError> {
    let mut user = repo::get(id)?; // ? converts RepoError → UserError automatically
    user.name = name.to_string();
    Ok(user)
}
```

The `?` operator handles propagation; `#[from]` handles wrapping; and a `match` on `UserError` is **exhaustively checked**. Add a new variant tomorrow and the compiler shows you every place that needs updating. And yet, error handling does not obscure the business logic. It's still easy to see what's going on and all of the situations where things can go wrong.

GENERIC IN GO ARE LEAKY ABSTRACTIONS

Go got generics in 1.18, and they're useful, but the implementation has constraints (no methods with type parameters, GC shape stenciling, occasional surprising performance characteristics). Rust generics monomorphize, which means each instantiation produces specialized code with zero runtime cost. Combined with traits, this gives you real zero-cost abstractions.

This matters less in handler code and more in shared infrastructure (middleware, generic repositories, decoders, parsers), where Go often pushes you back to `interface{}`/`any` plus type assertions.

LATENCY CONCERNS

Go's GC is excellent, concurrent, low-pause, well-tuned for typical service workloads. But "low-pause" is not "no-pause." Heavy memory pressure under load can cause P99 latency to spike while the Rust equivalent simply doesn't allocate on the hot path. That matters more often than many people would like to believe. P99 means that 1% of requests are slower than that number, and in a high-throughput service, that can be a significant number of requests. Even if the average latency is good, those outliers can lead to timeouts, unhappy customers, and cascading failures. Often, the most important routes or the routes which do the most data handling are affected. Once you are in such a situation, you'd typically have to heavily optimize and refactor your code (to reduce allocations, parallelize work, or offload to a separate service), which is hard.

I won't oversell this, for the vast majority of services, Go's GC is a non-issue. But for latency-sensitive systems (trading, real-time bidding, network proxies, high-throughput ingestion), the lack of GC pauses is a genuine selling point.

Go is great at our scale, but we really need something that is going to give us the price-per-

dollar performance capacity that we need, and Rust is going to get us there. That's why basically everything is heading towards Rust these days.

— Stephen Blum, CTO, PubNub, on Rust in Production

THE TRADEOFF

Go just optimizes for a different set of values than Rust, namely shipping speed and operational simplicity over compile-time guarantees. It's a design tradeoff.

Go is a very pragmatic language, but at a certain codebase size, the problems start to compound. Rust is worth it if the cost of shipping bugs exceeds the cost of a stricter compiler.

COMPARING BOTH LANGUAGES SIDE BY SIDE

The fastest way to feel comfortable in Rust is to map patterns you already know. Again, this can often feel like an apple-to-orange comparison, because solving the same problem in both languages often looks very different in practice. For a longer, fully-worked example of building the same backend service in both languages, see the [Shuttle comparison](#) [↗]. I do believe that there is value in looking at some code snippets side by side, just to get a feeling for the design decisions and patterns that come up most often.

ERROR HANDLING: `if err != nil` VS `Result<T, E>`

Go:

```
func ReadConfig(path string) (*Config, error) {
    data, err := os.ReadFile(path)
    if err != nil {
        return nil, fmt.Errorf("reading config: %w", err)
    }
    var cfg Config
    if err := json.Unmarshal(data, &cfg); err != nil {
        return nil, fmt.Errorf("parsing config: %w", err)
    }
    return &cfg, nil
}
```

Rust:

```
fn read_config(path: &Path) -> Result<Config, ConfigError> {
    let data = fs::read_to_string(path)?;
    let cfg = serde_json::from_str(&data)?;
    Ok(cfg)
}
```

The `?` operator collapses the error handling flow. Under the hood, it does the `if err != nil {`

`return err }` dance for you, including type conversion if necessary.

NULL: `nil` VS `Option<T>`

Go:

```
func GetUser(id string) *User {
    for _, u := range users {
        if u.ID == id {
            return &u
        }
    }
    return nil
}

u := GetUser("123")
fmt.Println(u.Name)
```

The caller has to remember to check for `nil` before dereferencing `u`. If they forget, they get a runtime panic. Rust doesn't have `nil`. If the absence of a value is a valid case, you'd use `Option<T>`:

```
fn get_user(id: &str) -> Option<User> {
    users.iter().find(|u| u.id == id).cloned()
}

let user = get_user("123");
println!("{}", user.name); // compile error: `user` is Option<User>, not User
// You must handle both cases:
match get_user("123") {
    Some(u) => println!("{}", u.name),
    None    => println!("not found"),
}
```

INTERFACES VS TRAITS

Go's interfaces are structural, a type satisfies an interface if it has the right methods.

```
type Reader interface {
    Read(p []byte) (n int, err error)
}
```

Initially, that looks very compelling, but it has some downsides. You can accidentally satisfy an interface without realizing it, and the compiler won't tell you when you add a new method to the interface that breaks existing implementers.

Rust's traits have to be implemented explicitly:

```
pub trait Reader {
    fn read(&mut self, buf: &mut [u8]) → std::io::Result<usize>;
}

impl Reader for MyType {
    fn read(&mut self, buf: &mut [u8]) → std::io::Result<usize> { /* ... */ }
}
```

The Go style is great for ad-hoc “duck typing.” Rust traits are explicit, so you can grep for every implementer. I do that a lot in practice.

The closest equivalent of `interface{}` / `any` in Rust is `Box<dyn Any>`, but you almost never want it. The Go community knows the cost of reaching for `interface{}` too:

interface{} says nothing.

— Rob Pike, [Go Proverbs](#) 

GOROUTINES VS ASYNC TASKS

Let me start by saying that I really like Go's concurrency model. It's as simple as adding `go` in front of a function call, and the runtime picks it up and runs it as a green thread:

```
go doWork(ctx, input)
```

In combination with channels, that's a true superpower.

Don't communicate by sharing memory; share memory by communicating.

— Rob Pike, [Go Proverbs](#) 

In Go there is no syntactic distinction between sequential and parallel code. Any function can be called normally, dropped into a `go` statement, or invoked from inside a goroutine, without changing its signature, its callers, or anything about how it's written. There is no `async fn`, no `.await`, no executor to pick, no `Send`/`Sync` bounds to satisfy.

Sequential and concurrent code is identical as long as you don't share mutable state without synchronization.

That property, the absence of *function colouring*, is the single biggest day-to-day productivity win Go has over Rust, and it's the thing Go developers miss most after switching. Rust async is more powerful, but it's also more explicit in your code, and that visibility has a real ergonomic cost.

Rust uses `async`/`await` on top of an executor (almost always `tokio` for backend services):

```
tokio::spawn(async move {
    do_work(input).await;
```

```
});
```

- Rust async functions return `Future`s. They don't run until awaited or spawned.
- The compiler tracks `Send`/`Sync` across `.await` points. If you hold a non-`Send` value across an await, you get a compile error explaining exactly why.
- There's no built-in goroutine-style preemption. Long CPU-bound work in an async task starves the executor; you offload to `tokio::task::spawn_blocking` or `rayon` instead.
- Channels (`tokio::sync::mpsc`, `broadcast`, `watch`) are first-class but get developed as crates, not as part of the language.

You might walk away from this section, thinking that Go's concurrency model is objectively better, but Go's model is not without sin, either:

- `WaitGroup` and `sync.Once` are easy to misuse, leading to goroutine leaks or deadlocks.
- Until Go 1.14 the scheduler was cooperative; it now preempts goroutines asynchronously, so a tight CPU-bound loop no longer starves the system the way it used to.³
- Go's `context.Context` is a great convention for cancellation, but it's easy to forget to pass it through every call site. The compiler does not check that for you.
- Managing shared mutable state with `sync.Mutex` and `sync.RWMutex` is error-prone. Missing a lock only fails at runtime.

Go doesn't have a way to tell a goroutine to exit. There is no stop or kill function, for good reason. If we cannot command a goroutine to stop, we must instead ask it, politely.

— Dave Cheney, [The Zen of Go](#) ↗

In Go that "asking politely" is a `context.Context` plumbed through every call site by convention. In Rust it's a `CancellationToken` (or a `watch` channel) plumbed through every call site, but the compiler can actually tell you when you forgot.

Then again, none of that truly matters in practice. For most backend code, the day-to-day feel is similar: spawn a task, communicate via channels, use timeouts liberally.

And to be fair, Go 1.25 added `WaitGroup.Go` ↗, which turns the `Add(1)`/`go`/`defer Done()` dance into a single call, which removes the most common way to get the counter wrong:

```
func main() {
    wg := new(sync.WaitGroup)
    wg.Go(foo)
    wg.Go(bar)
    wg.Wait()
}
```

That's really nice, but not everyone knows about it yet.

CHANNELS

Both languages have channels.

```
ch := make(chan int, 10)
go func() {
```

```
ch ← 42
}()
v := ←ch
```

```
let (tx, mut rx) = tokio::sync::mpsc::channel::<i32>(10);
tokio::spawn(async move {
    tx.send(42).await.unwrap();
});
let v = rx.recv().await.unwrap();
```

Both are pretty straightforward, but Rust's channels distinguish sender and receiver as separate types, which makes ownership and `Send`-ness explicit at the type level.

STRUCTS AND METHODS

Go:

```
type Circle struct {
    Radius float64
}

func (c Circle) Area() float64 {
    return math.Pi * c.Radius * c.Radius
}
```

Rust:

```
pub struct Circle {
    pub radius: f64,
}

impl Circle {
    pub fn area(&self) → f64 {
        std::f64::consts::PI * self.radius * self.radius
    }
}
```

Rust's `&self` is the equivalent of a Go value receiver; `&mut self` is a pointer receiver with mutation. Owned `self` (consuming the value) has no Go analog and is occasionally very useful (tpestate, builders).

STRINGS: string VS String AND &str

Go only has a single string type. While you can store any sequence of bytes, they *commonly* represent UTF-8 encoded text.

From this description, you can already see the problem: data validation is left as an exercise for the programmer. You can have a `string` that contains totally invalid UTF-8, and the compiler won't stop you. This leads to a lot of surprising edge cases:

First Go, then Rust:

```
s := "héllö"
s[1] // → 0xC3 (a byte)
```

```
let s = "héllö";
s[1] // compile error:
      // `str` cannot be indexed
      // by `{integer}`
```

Note how the character `é` is two bytes in UTF-8, so `s[1]` is not the second character, but the first byte of the second character. Our human intuition for what a “character” is often doesn't match the underlying byte representation. We see the `é` as one character, but it's actually two bytes (`0xC3 0xA9` in UTF-8). ⁴

Rust has a lot of string types, which allow you to say *exactly* what you mean and get the compiler to enforce it. The two main ones are:

- `String`, owned, heap-allocated, growable. Equivalent to `[]byte` you intend to mutate.
- `&str`, a borrowed view into someone else's string data. Equivalent to a Go `string` *parameter* most of the time.

As a rule of thumb, take `&str` in arguments, return `String` when you produce new data.

```
fn greet(name: &str) → String {
    format!("Hello, {name}")
}
```

This is mostly painless once you internalize it.

In practice, strings are very complicated organisms. Go uses strings for a lot of different concepts, such as file paths, URLs, byte buffers, etc., and that can lead to a lot of confusion and incorrectness. In Rust, the cost gets pushed to the development phase. That is quite painful in the beginning, but it helps to clear up those misconceptions upfront about how strings are supposed to behave vs. how they actually behave on the system level.

After using Rust for many years, I can't imagine going back to a world where strings are “a bunch of bytes thank you very much have fun.” I have PTSD from the many times I've had to debug encoding issues and input handling which stemmed from the fact that I couldn't express the guarantees of “which kind of string” I was dealing with.

GO GENERICS ARE TOO LITTLE, TOO LATE



Skip this section if you don't care about generics much. 😊 In hindsight, I don't think it matters for most working engineers, but it explains part of the design split between Go and Rust.

Go got generics in 1.18 (March 2022), thirteen years after the language shipped. They are useful, but they feel tacked on, and in practice they have most of the *downsides* of a generic type system without delivering the *upsides* you'd expect coming from Rust, Haskell, or even modern C++.

This is a very strong claim, so let me back it up.

THE STANDARD LIBRARY BARELY USES THEM

The most telling signal is that three years after generics landed, Go's own standard library still mostly avoids them. `sort.Slice` still takes a `func(i, j int) bool` closure instead of a `cmp.Ordered` constraint. `sync.Map` is still typed as `any/any`. The generic helpers that *do* exist live in a small handful of packages: `slices`, `maps`, `cmp`, and a few entries under `sync`.

It's fair to point out that backwards compatibility is part of the story here: the Go 1 compatibility promise means the existing non-generic APIs can't be retrofitted, so any generic version has to live alongside them (or in a new package). But that's only part of the explanation. Three years is plenty of time to introduce generic alternatives, and the fact that very few have appeared suggests the language designers don't lean on generics as a primary tool the way Rust does.

Compare that to Rust, where generics permeate the standard library from day one: `Option<T>`, `Result<T, E>`, `Vec<T>`, `HashMap<K, V>`, `Iterator`, `From/Into`, `AsRef`, `Borrow`, every collection, every smart pointer. You cannot write idiomatic Rust without using generics, because the standard library *is* generic.

In Go, generics are an opt-in feature for library authors who really need them. In Rust, they're the substrate everything else is built on.

NO TRAIT SYSTEM, JUST STRUCTURAL CONSTRAINTS

Rust's generics are tied to traits, which double as the language's mechanism for ad-hoc polymorphism, supertraits, associated types, blanket impls, and coherence.

Go's constraints are just interfaces with an extra `~` operator for type-set membership. There are no:

- Rust has supertraits and constraint hierarchies. You can write `trait Ord: Eq + PartialOrd`, and any `T: Ord` automatically satisfies `Eq` and `PartialOrd`. Go has interface embedding, but the constraint solver does not reason about hierarchies the way Rust's trait system does.
- Rust has associated types. `Iterator` has `type Item;`, so `T::Item` is a first-class thing you can name in bounds. Go's closest equivalent is a second type parameter, which leaks into every signature.
- Rust has blanket impls. `impl<T: Display> ToString for T` automatically gives every `Display` type a `to_string()` method. Go has no way to add methods to a type from outside its defining package, generic or not.

- Rust supports methods with their own type parameters. Go has long treated this as an explicit, documented [↗](#) non-feature. You cannot write `func (s Set[T]) Map[U](f func(T) U) Set[U]`⁵. In Rust, generic methods on generic types are routine.

The practical consequence is that the moment your abstraction needs more than “a function that works for any `T` with these few operations,” Go pushes you back to `any` plus type assertions, code generation, or runtime reflection.

TYPE INFERENCE STOPS AT THE FUNCTION BOUNDARY

Rust uses a Hindley-Milner-style inference engine that propagates type information through entire expressions, including across closures, iterator chains, and `?` operators. You routinely write:

```
let evens: Vec<_> = (0..100).filter(|n| n % 2 == 0).collect();
```

and the compiler figures out `_` is `i32` from the range, and `Vec<_>` is `Vec<i32>` from the `collect` target.⁶

Go’s inference is much shallower. It can usually infer type parameters from function arguments, but it cannot infer from return-position context [↗](#), cannot chain inference through generic builders the way Rust does, and frequently forces explicit type arguments at call sites:

```
result := slices.Collect[int](iter) // often required
```

In Rust this is the exception; in Go it’s still common.

MONOMORPHIZATION VS GC SHAPE STENCILING

There’s no free lunch with generics; you either pay at compile time, at runtime, or you give up specialization (more on that in a bit). C++ and Rust pay at compile time through monomorphization. Java pays at runtime through type erasure plus the JIT. Go picked a middle path with GCShape stenciling and dictionaries [↗](#): types that share a “GC shape” share the same compiled function and dispatch through a runtime dictionary.

The Go choice keeps compile times fast, which is a real and valuable property. The cost is that generic Go code can be measurably *slower* than the equivalent hand-written non-generic version, because every method call on a type parameter goes through an indirection. There’s a well-known PlanetScale post [↗](#) showing exactly this.

Rust monomorphizes, which means every `Vec<i32>` and `Vec<String>` produces specialized machine code with zero runtime dispatch. Generic code is the *fast* path, and reaching for `dyn Trait` (the equivalent of Go’s interface dispatch) is a deliberate choice you make when you want runtime polymorphism. You pay for monomorphization with compile times, which is the same bill C++ has been paying for decades. Neither tradeoff is obviously right; they just optimize for different things.

THEY DON’T PLASTER OVER HOLES IN THE TYPE SYSTEM

This is the part that bothers me most.

A good generics system *removes* reasons to fall back to escape hatches. In Rust, generics + traits eliminate most of what you'd otherwise need `Box<dyn Any>` or runtime reflection for. The type system gets stronger.

In Go, generics did not remove `any`, `reflect`, or code generation as the dominant pattern for ORMs and decoders. Mocks still often rely on generated code. `encoding/json` still uses reflection. `database/sql` still uses `any`. The places where a real generics system would shine are the same places Go reaches for runtime mechanisms it had before 1.18.

Generics in Go feel additive, a new tool in the box that's useful in narrow cases. Generics in Rust feel foundational; remove them and the language collapses.

That's the difference, and it's why generic Go code, in my experience, doesn't read better than the `interface{}`-based code it replaced; it just reads differently, with more punctuation.

But I would also like to acknowledge that all of this doesn't matter for 95% of code out there. The different perspective on generics is a philosophical one: In Rust, they are an integral part of the language's design, and it's normal to use them to model behavior. In Go, they're tacked on and meant for the 5% edge-cases in library code which are otherwise just painful to write.

POPULAR GO PACKAGES AND THEIR RUST COUNTERPARTS

A lot of what you'd pull a crate for in Rust ships out of the box in Go, such as `net/http`, `encoding/json`, `database/sql`, `log/slog`, `testing` + `httptest`, and plenty more. Rust's stdlib is smaller by design. The language ships the core and lets the ecosystem evolve the rest. (The canonical example is random number generation, which lives in the `rand` crate, not `std`.)

That said, Go's stdlib does carry some legacy. For example, `math/rand` was effectively re-released as `math/rand/v2` in Go 1.22 [↗](#) to fix non-backwards compatible issues with the generator. Previously, simply calling `rand.Seed` would risk not getting truly random numbers [↗](#).

With that out of the way, here's a rough map. Entries marked (stdlib) are part of Go's standard library:

Concern	Go	Rust
HTTP server	<code>net/http</code> (stdlib) (+ <code>chi</code> , <code>gin</code> , <code>echo</code>)	<code>axum</code> (on <code>hyper</code>)
HTTP client	<code>net/http</code> (stdlib) (+ <code>resty</code>)	<code>request</code>
gRPC	<code>google.golang.org/grpc</code> + <code>protoc-gen-go</code>	<code>tonic</code> + <code>prost</code>
SQL	<code>database/sql</code> (stdlib) (+ <code>sqlc</code> , <code>sqlx</code> , <code>gorm</code>)	<code>sqlx</code> , <code>sea-orm</code> , <code>diesel</code>
Migrations	<code>golang-migrate</code> , <code>goose</code>	<code>sqlx migrate</code> , <code>refinery</code>
JSON	<code>encoding/json</code> (stdlib) (+ <code>sonic</code> , <code>goccy/go-json</code>)	<code>serde</code> + <code>serde_json</code>
Logging	<code>log/slog</code> (stdlib) (+ <code>zerolog</code> , <code>zap</code>)	<code>tracing</code> + <code>tracing-subscriber</code>
Metrics	<code>prometheus/client_golang</code>	<code>metrics</code> + <code>metrics-exporter-prometheus</code>
Config	<code>viper</code> , <code>koanf</code>	<code>figment</code> , <code>config</code> (config-rs)

Concern	Go	Rust
CLI	<code>flag</code> (stdlib) (+ <code>cobra</code> , <code>urfave/cli</code>)	<code>clap</code> (derive)
Errors	<code>errors</code> (stdlib) + <code>fmt.Errorf("...%w", err)</code>	<code>thiserror</code> (libraries), <code>anyhow</code> (binaries)
Testing	<code>testing</code> + <code>httptest</code> (stdlib) (+ <code>testify</code>)	built-in <code>#[test]</code> , <code>rstest</code>
Mocking	<code>uber-go/mock</code> (maintained fork of <code>mockgen</code>), <code>moq</code>	hand-written fakes (idiomatic), <code>mockall</code>
HTTP test fakes	<code>httptest</code> (stdlib)	<code>wiremock</code> , <code>httpmock</code>
Test containers	<code>testcontainers-go</code>	<code>testcontainers</code>
Random numbers	<code>math/rand/v2</code> (stdlib)	<code>rand</code> crate
Background tasks	<code>goroutines</code> + <code>errgroup</code>	<code>tokio::spawn</code> + <code>JoinSet</code>

For a typical backend service in Rust, `axum` + `sqlx` + `tokio` + `tracing` + `serde` + `clap` covers about 90% of what you need. The dependency count is higher than the Go equivalent, and that’s a real cost (more `Cargo.lock` churn, more supply-chain surface). For services where the stdlib is enough, staying on Go is a perfectly defensible call.

KEY CHALLENGES IN TRANSITIONING TO RUST

I want to be straightforward here. Coming from Go, **you will hit a wall**. The wall has a name.

THE BORROW CHECKER

Go’s runtime handles memory and aliasing for you. Rust pushes that decision into the type system. The first few weeks you’ll write code that “should obviously work” and the compiler will refuse it. There’s this old joke:

What’s the difference between a Rust beginner and an expert?
A beginner asks: “Why does the compiler stop me from doing things? This is horrible!”
An expert asks: “Why doesn’t the compiler stop me from doing things? This is horrible!”

Here are the most typical ways the borrow checker gets in your way initially:

1. Long-lived references. In Go, you’d happily hold a `*User` from a map for as long as you want. In Rust, that borrow blocks mutation of the map for its whole lifetime. The fix is usually to clone, or to scope the borrow tighter.
2. Self-referential structs. This is common in Go: a struct holding both data and an iterator over it. In Rust, this requires `Pin`, `ouroboros`, or a redesign. Almost always: redesign.
3. Shared mutable state across goroutines. What you’d write as `mu sync.Mutex; data map[K]V` becomes `Arc<Mutex<HashMap<K, V>>>`. Slightly more verbose, much more checked.
4. Returning references from functions. Lifetime annotations show up. They’re not as bad as their reputation, but they’re a new concept. Think of them as types that describe how long

something lives inside your program.

With all of these rules, the borrow checker truly sounds like a “gatekeeper” of sorts, which keeps getting in the way and is just overall frustrating to deal with. That is not the mental mindset you should have when learning Rust! **The borrow checker uncovers real bugs that already exist in your code**, and if you don’t address them, your program will have safety issues. It’s a common misconception that the borrow checker makes things harder, whereas in actuality all the problems have existed before, but the borrow checker is the first and only thing that points them out.

So whenever you get a compiler error from `rustc`, take a step back and think how your code could break. A few questions you can ask yourself:

- If a value *got moved* from one place to another, can the original place still use it safely?
- If a value *is shared* across threads, can one thread modify it while another thread reads it?
- If a pointer *is dereferenced*, can it be null or dangling?
- When a value *goes out of scope*, can anything still refer to it?

That is the mindset you need to understand the borrow checker.

Humans are bad at reasoning about memory. We forget that pointers can be null. Old references can outlive the data they point to. Multiple threads can touch the same data at the same time. We tend to have a “linear” mental model of how data flows through a program, but in reality it’s closer to a complex graph with many paths and interactions. Every `if` condition forces you to consider what happens in *both* branches. Every loop forces you to consider what happens on *every* iteration. That is exactly the kind of reasoning the borrow checker is designed to do for you! It enforces best practices at compile time, and it can feel annoying when your own mental model disagrees with the borrow checker’s (which is the more accurate one 99% of the time). There *are* cases where the borrow checker is genuinely too strict, but they are rare, and as a beginner you’ll almost never run into them. I got memory management wrong plenty of times in my early days, but I approached it with a *learner’s mindset*, which helped me ask “what’s wrong with my code?” instead of “what’s wrong with the compiler?”, a reaction I see a lot in trainings.

The good news is that once you internalize borrowing, it stops fighting you. Most experienced Rust developers will tell you that the borrow checker is like having a very attentive programming companion that really cares about memory safety.

The first month is the hardest.

When you started to get into it: frustration. It reminded me of what it was like to learn programming for the first time, because it's so different. With the borrow checker and lifetimes, I didn't want to have to deal with those things — but I was forced to.

— Stephen Blum, CTO, PubNub, on [Rustacean Station](#) ↗

And here’s Ed Page (maintainer of `clap`) on the other side of that curve, which is what you should be optimizing for:

The borrow checker has saved me from having to think about these problems, and instead I'm able to focus on higher-level problems. It's helped catch things when I've done my own analysis and failed at it.

— Ed Page, on [Rustacean Station: clap with Ed Page](#) ↗

COMPILE TIMES

Be honest with your team: Rust compile times are a real downgrade from Go's.

A clean release build of a medium service can take minutes in comparison to Go's near-instantaneous compiles. Incremental builds and `cargo check` are reasonable and compile times have gotten much better over the years, but you'll feel the difference.

Then again, you get a lot of additional compile-time checks in return.

To improve compile times, use `cargo check` in your edit loop, split into a workspace once it pays off, and keep proc-macro-heavy crates in their own crate so they only recompile when they change. See [tips for faster Rust compile times](#) for a deeper dive.

In practice, compile times are rarely a problem for me anymore. Modern laptops are plenty fast and the toolchain has improved a lot in the last few years.

ASYNC COLORING

As covered in [Goroutines vs Async Tasks](#), Rust's `async fn` / `fn` split is one of the biggest ergonomic regressions coming from Go. Async traits have been stable since Rust 1.75, but there are still rough edges around mixing them with dynamic dispatch; occasionally you'll reach for the `async-trait` crate to paper over them.

SMALLER ECOSYSTEM IN SOME NICHES

Rust's crate ecosystem is growing and libraries are high-quality across the board, but Go has a head start in some backend-adjacent domains: Kubernetes operators, cloud-provider SDKs, database drivers for certain niche stores. Before you commit, spend a day checking that the libraries you depend on have Rust equivalents you're willing to use. Teams I help often have to hand-roll at least one or two core libraries themselves. For example, they might have to update an abandoned crate for XML schema validation, or write their own client for a lesser-known protocol.

INTEGRATION STRATEGIES

You don't have to rewrite everything in one go. Every successful Go-to-Rust story I've heard so far was very methodical. Victor Ciura from Microsoft put it well:

We're not madly going across the board and just, for the fun of it, rewriting everything in Rust. We're doing these tactical choices where we say: okay, this new component, it's better if we [do it in Rust].

— Victor Ciura, Principal Engineer, Microsoft, on [Rust in Production](#)

There are a few tried and true strategies for the migration:

1. CARVE OFF A HOT PATH AS A SERVICE

If one specific service in your fleet is the perpetual problem child (high CPU, latency-sensitive,

or constantly hit with reliability issues), rewrite *just that one* in Rust, behind the same API contract. Other Go services keep talking to it via HTTP/gRPC, oblivious to the underlying language. Being able to do this can be super motivating:

If you go on Hacker News and look up “migrating to Rust,” the first result is always this one about [Discord moving from Go to Rust](#). It almost motivated us to see [if we could do the same].

— Jeff Kao, CTO, Radar, on [Rust in Production](#)

2. REPLACE A SIDECAR / WORKER PROCESS

Background workers, queue consumers, ingestion pipelines, and CPU-bound batch jobs are excellent first targets. They typically have a clear input/output boundary (a queue, a topic) and no shared in-process state with the rest of the system.

3. CGO IS POSSIBLE BUT PAINFUL

You *can* call Rust from Go via cgo, and [there are guides on how to do it](#). (Reach out if you’d be interested in a guide on this from me.) In practice, I rarely recommend it for backend services. The build complexity and FFI overhead usually outweigh the benefits compared to “just stand up a Rust service and put it behind a network call.” For libraries and CLI tools, it’s more viable.

4. STRANGLER PATTERN BEHIND A GATEWAY

If you have an API gateway or reverse proxy, you can route specific endpoints to a new Rust service while the rest stays in Go. This works particularly well when one bounded context (auth, search, billing) is the right unit to migrate. The pattern is often called “[strangler fig](#),” because the new service grows around the old one until it eventually replaces it entirely.

Practical Migration Tips

Start with a service that has a clear boundary. Don’t pick the most central, most-deployed service in your fleet. Pick the one where the contract with the rest of the system is well-defined and the blast radius is small.

Keep the same API contract. If your Go service exposes a REST API, your Rust service should too: same paths, same JSON shapes, same error envelope. The migration is invisible to clients, and you can swap traffic incrementally with a gateway.

Don’t translate idioms verbatim. Resist the urge to write Go-flavoured Rust. `if err != nil { return err }` becomes `?`. Goroutine-per-request becomes `tokio::spawn` only when you actually need it (axum already concurrently handles requests). Interfaces with one method usually become trait bounds on a generic instead of `Box<dyn Trait>`.

Use the compiler as a pair programmer. Rust’s compiler errors are usually pretty good. Read them slowly. They almost always tell you the right answer. The team members who struggle longest are the ones who fight the compiler instead of treating it as a

collaborator.

Invest in training early. I've seen teams try to do a Rust migration "on the side," learning as they go. It rarely ends well. It's a bit like training for a marathon by signing up for the race and then trying to run it without any prior training. You can do it, but it's going to be painful and you might not finish.

Block off real time for learning: a workshop, [an online course](#) [↗], paired sessions on real code. The upfront investment pays back many times over once the team is fluent. (Hey, if you want to talk about training options, [I'm happy to chat.](#))

KEEPING GO'S STRENGTHS

Not everything should be migrated. Go is excellent for Kubernetes-native tooling such as operators and controllers, where the ecosystem is overwhelmingly in Go. It's also a great fit for CLI utilities and dev tooling, thanks to its fast compiles, easy cross-compilation, and simple deployment story. Glue services like thin API layers and format converters are another sweet spot, since the boilerplate ratio in Rust isn't worth it for that kind of work. More broadly, Go shines anywhere your team velocity matters more than absolute correctness guarantees.

Go is a very fine choice for networking services. We have a lot of Go at Canonical — Juju is a huge Go codebase.

— Jon Seager, VP of Engineering, Canonical, on [Rust in Production](#)

A hybrid strategy is fine and common. Many of the teams I work with end up with a polyglot backend: Go for the "boring" services, Rust for the ones where reliability and performance pay back the extra effort.

EXPECTED IMPROVEMENTS

Numbers vary wildly by workload, so take these as rough guidance. Not promises! But here are some ballpark numbers, based on Go-to-Rust migrations I've helped with:

- Production incidents: this is the one teams are most surprised by. The classes of bugs that survive `go test -race` and reach production (data races, nil dereferences, missed error paths) just don't compile in Rust. Oncall rotations are typically very boring after a Rust migration.
- CPU usage: 20-40% improvement. Less dramatic than Python-to-Rust, because Go is already efficient. The wins come from no GC and tighter loops.
- Memory: 30-50% reduction, mostly from the absence of GC overhead and a smaller runtime.
- P99 latency: more consistent under load. Rust services tend to flatline where Go services can show GC-induced jitter. This has improved on the Go side since the low-latency GC work, but the difference can still show up under heavy load.

I hadn't had to chase down a crash, or some weird multi-threaded race condition, or some of these other things which actually consumed a huge amount of my time before.

— *Andrew Lamb, Staff Engineer, InfluxData, on [Rustacean Station: Rebuilding InfluxDB with Rust](#)*



Honestly, you're unlikely to get a 10x throughput improvement. What you get is fewer "silly errors" and flatter latency tails, plus the ability to expand into other domains like embedded development or systems programming while still using the same language. That's often the most surprising side-effect of a migration: there's a lot of opportunity for code-sharing across teams that previously had to use different stacks. You can use Rust for everything.

CONCLUSION

Going from Go to Rust is a different kind of migration than coming from [Python](#) or [TypeScript](#). Coming from Go, you already know the benefits of a statically-typed, compiled language. You're likely looking for a codebase with fewer footguns and a stricter compiler that catches more mistakes at compile time.

For [foundational services](#) (services that your organization relies on, that have high uptime requirements, that are critical to your business), that trade is obviously worth it. For others, Go is fine. The point of a migration is to put each problem in the language that solves it best.



Ready to Make the Move to Rust?

I help backend teams evaluate, plan, and execute Go-to-Rust migrations. Whether you need an architecture review, training, or hands-on help porting a critical service, [let's talk about your needs](#).

1. You *can* still opt out with `.unwrap()` or `.expect()`, which panic on `None`. The difference is that those calls are explicit and easily greppable. They stand out in review, unlike an implicit nil dereference that compiles but fails at runtime. When one panic does slip through it tends to make the news: [Cloudflare's November 2025 outage](#) was traced to a Rust `.unwrap()` that panicked on an unexpectedly large generated file. In Rust, panics are not impossible, but ignoring absence has to be a deliberate, visible choice.
2. To be precise: *safe* Rust eliminates data races by construction, a value that can't be shared across threads without synchronization simply won't compile. It does *not* prevent race conditions in the broader sense (deadlocks, livelocks, or logic bugs in your synchronization); no type system does. What goes away is the "oh no, I forgot to lock this" class of silent data corruption. Read more [here](#).
3. Before Go 1.14, the scheduler only switched goroutines at function-call safe points, so a tight loop with no calls could hog a thread indefinitely. Go 1.14 introduced signal-based [asynchronous preemption](#) , which fixes that.
4. Worth disambiguating, since it trips people up: a Go `string` is an immutable sequence of *bytes* that is conventionally (but not guaranteed to be) valid UTF-8. A `rune` is a Unicode code point (an alias for `int32`), what you get when you range over a `string`. `[]byte` is the mutable byte buffer. The closest one-to-one mapping is `string` (Go) $\leftrightarrow$ `&str` (Rust) for read-only views, and `[]byte` (Go) $\leftrightarrow$ `Vec<u8>` (Rust) for mutable buffers. `String` in Rust is the owned, growable version of `&str`, with the additional guarantee that its contents are valid UTF-8 (which Go's `string` does *not* enforce at the type level). For more information, see [Strings, bytes, runes and characters in Go](#) .
5. To be precise, this is about methods that introduce *their own* type parameters beyond the receiver's. Go has had generic *functions* and generic types since 1.18, so `func Map[T, U any](s []T, f func(T) U) []U` is fine. What you can't do *today* is attach that `Map` to a method on a generic `Set[T]` and let the caller pick `U` per call. That

restriction is on its way out: the [generic methods proposal](#)  was accepted in early 2026 and is targeted at Go 1.27. 

6. If you're coming from Go, that line takes a minute to parse: `(0..100)` is a lazy range, `.filter(|n| ...)` is a closure (the `|n|` is the parameter list, no curly braces needed for a single expression), and `.collect()` materializes the iterator into whatever type the left-hand side asks for. Go does not lean on this style, while idiomatic Rust uses it often. You can still write a `for` loop in Rust, and for one-off code that's often the right call. [Iterator chains start to feel natural after a while](#), and chaining transformations without intermediate variables can make the data flow easier to read. 

ADDITIONAL RESOURCES

- [JetBrains State of Developer Ecosystem 2024](#) : source for the Go usage figures cited in this guide
- [Rust vs Go: A Hands-On Comparison \(Shuttle\)](#) : a longer, code-heavy comparison I wrote with the Shuttle team
- [Rust Prevents Data Races, Not Race Conditions](#): what Rust's concurrency guarantee actually covers, and what it doesn't



Published: 2026-05-21

Last updated: 2026-07-20

[Revision history](#)

Author: [Matthias Endler](#)

Editor: [Simon Brüggen](#)

Reviewers: John Souvestre

IDIOMATIC RUST CONTENT. STRAIGHT TO YOUR INBOX.

I regularly write new articles on idiomatic Rust. If you want to be notified when I publish them, you should sign up to my newsletter here. No spam. Unsubscribe at any time.

mail@example.com

Subscribe

[↑Back to top](#)

IDIOMATIC RUST IN YOUR INBOX

Occasional articles and podcast updates. No spam, unsubscribe any time.

mail@example.com

Subscribe

OFFERINGS

For Teams

For Individuals

RESOURCES

Blog

Podcast

Workshops

Why Rust?

Case Studies

Conferences

Learning Resources

TOOLS

Learn Rust

Rust Search

Rust Tool Index

MIGRATION GUIDES

Python

TypeScript

Java

Scala

C++

COMPANY

GitHub

LinkedIn

Legal Notice